



DESIGN PATTERN IMPLEMENTATION



SINGLETON PATTERN

INTRODUCTION:

The singleton pattern ensures that only one object(instance) of a class exists throughout the application and provides a global access point to that instance.

The Singleton Pattern is a creational design pattern used when we want to make sure that only one object of a particular class exists throughout the application. Instead of creating a new object every time it is needed, the same object is shared and reused.

This pattern is useful when having multiple objects could cause problems or unnecessary resource usage. Common examples include database connections, configuration managers, logging systems, and application settings.

PROBLEM:

Without the Singleton Pattern, a program may create multiple objects of the same class even when only one object is actually needed. This can lead to unnecessary memory usage, inconsistent data, and difficulty in managing shared resources.

Some common problems are:

- **Multiple objects:** Different parts of the application may create separate objects of the same class.
- **Wasted resources:** Creating multiple objects can consume unnecessary memory and processing power.
- **Inconsistent data:** Different objects may contain different values, causing unexpected results.
- **Difficulty managing shared resources:** Resources such as database connections, configuration settings, or logging systems can become difficult to control.
- **No centralized control:** There is no single object responsible for managing a resource that should be shared across the application.

Why we need only one instance of an object:

- Thread poles
- Caches
- Dialog boxes
- Objects used for logging
- Objects that handle graphics cards.

SOLUTION:

The Singleton Pattern solves these problems by restricting a class to create only one object and providing a common way to access that object whenever it is needed.

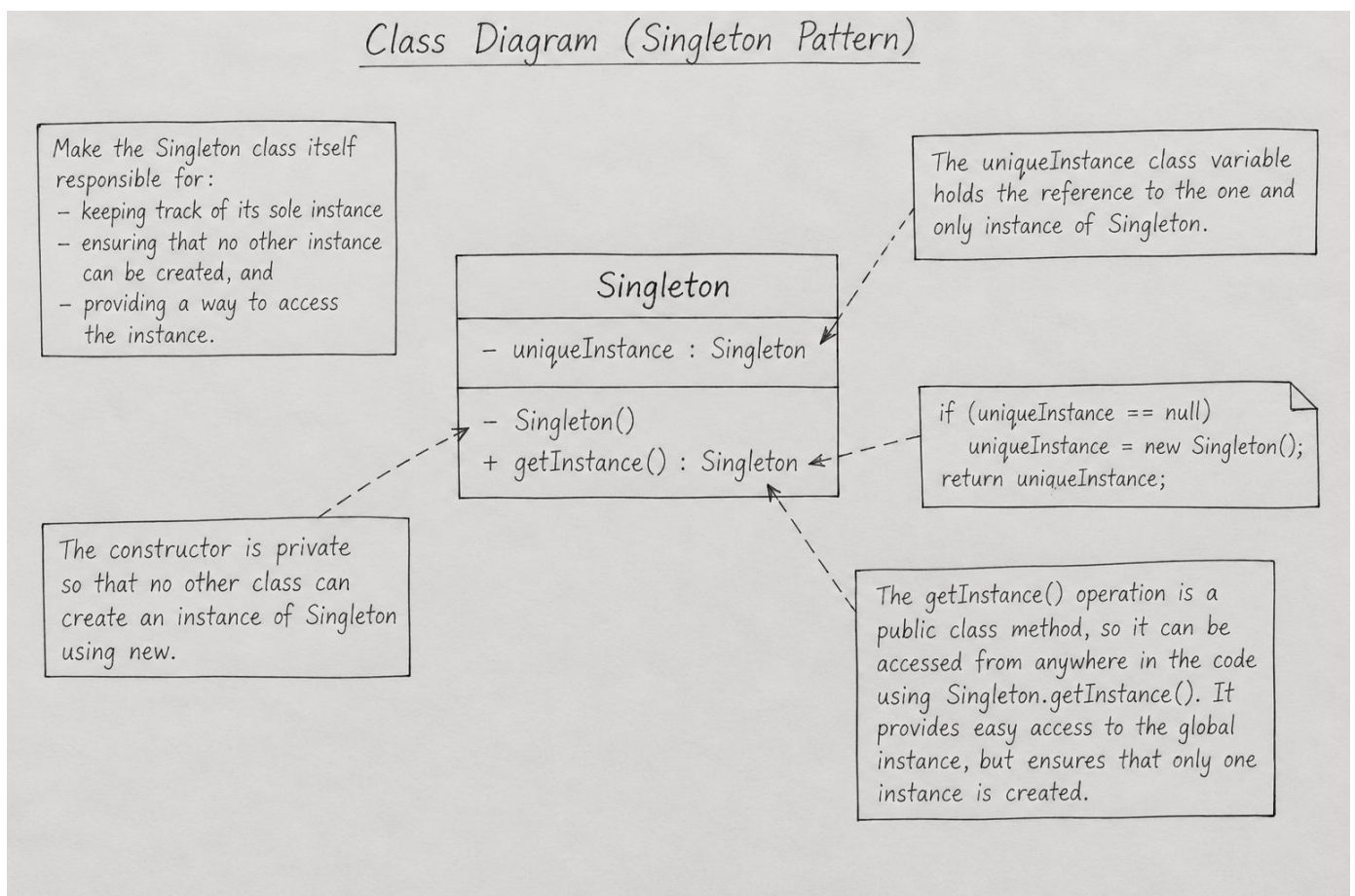
The class itself controls the creation of its object. If an object already exists, it does not create another one; instead, it returns the same existing object.

A global variables can make an object accessible.

This provides:

- **Single object:** Only one instance of the class exists.
- **Resource efficiency:** Prevents unnecessary object creation and resource usage.
- **Consistent data:** All parts of the application work with the same object and its data.
- **Centralized control:** A shared resource can be managed from one place.
- **Easy access:** Different parts of the application can access the same instance when required.

CLASS DIAGRAM:



REAL-WORLD EXAMPLE: DATABASE CONNECTION:

A common real-world application of the Singleton Pattern is managing a database connection. In an application such as a food-ordering website, many different components need to communicate with the same database. For example, the login system may need to check user details, the order system may store orders, and the payment system may update transaction information.

If each component creates its own database connection, it can consume more memory and system resources and may make the application difficult to manage. Instead, the Singleton Pattern creates one database connection object and allows all the required components to use that same connection.

For example:

Food Ordering Website

↓

Single Database Connection

↓

Login System → Order System → Payment System → User Management

This ensures that there is only one shared connection, reducing unnecessary resource usage and providing centralized control over the database connection.

IMPLEMENTATION:

The Singleton Pattern is implemented by ensuring that a class can create only one object during the entire execution of the application. The class itself controls the creation and access of this object.

The implementation of Singleton Pattern mainly consists of the following steps:

- **Create a private static instance:**

A static variable is created inside the class to store the single object of that class. Initially, it is set to null.

- **Make the constructor private:**

The constructor is declared as private. This prevents other classes from directly creating objects using the new keyword.

- **Create a public static getInstance() method:**

This method is used to access the single object. Since it is static, it can be called directly using the class name without creating an object first.

- **Check whether the object already exists:**

Inside getInstance(), the program checks whether the instance is null.

- If it is null, a new object is created.
- If it already exists, the same object is returned.

- **Reuse the same object:**

Every subsequent call to `getInstance()` returns the previously created object instead of creating a new one.

→ **JAVA implementation:**

```
class DatabaseConnection {

    // Stores the single instance of the class
    private static DatabaseConnection instance;

    // Private constructor prevents object creation
    // from outside the class
    private DatabaseConnection() {
        System.out.println("Database Connection Created");
    }

    // Public method to provide access to the single instance
    public static DatabaseConnection getInstance() {

        // Create the object only if it does not already exist
        if (instance == null) {
            instance = new DatabaseConnection();
        }

        // Return the existing object
        return instance;
    }

    // A method representing database operations
    public void connect() {
        System.out.println("Connected to the database.");
    }

    public void executeQuery(String query) {
        System.out.println("Executing Query: " + query);
    }
}
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        // First request for the database connection
```

```
        DatabaseConnection db1 =
```

```
            DatabaseConnection.getInstance();
```

```
        db1.connect();
```

```
        db1.executeQuery("SELECT * FROM users");
```

```
        // Second request for the database connection
```

```
        DatabaseConnection db2 =
```

```
            DatabaseConnection.getInstance();
```

```
        db2.executeQuery("SELECT * FROM orders");
```

```
        // Check whether both references point
```

```
        // to the same object
```

```
        if (db1 == db2) {
```

```
            System.out.println("Both references use the same object.");
```

```
        } else {
```

```
            System.out.println("Different objects were created.");
```

```
        }
```

```
    }
```

```
}
```

OUTPUT: Database Connection Created

Connected to the database.

Executing Query: SELECT * FROM users

Executing Query: SELECT * FROM orders

Both references use the same object.

How it works:

When `DatabaseConnection.getInstance()` is called for the first time, instance is null, so the program creates a new `DatabaseConnection` object.

When `getInstance()` is called again, the object already exists. Therefore, the program does not create another object and simply returns the existing one.

In the example, `db1` and `db2` both refer to the same database connection object. The `db1 == db2` check confirms this.

This implementation is useful when an application needs a single shared resource, such as a database connection, logger, configuration manager, or application settings.

PROS AND CONS OF SINGLETON PATTERN:

Pros

- **Controls object creation:** It ensures that only one instance of a class is created throughout the application.
- **Saves resources:** Since the same object is reused, it can reduce unnecessary memory usage and object creation.
- **Easy global access:** The single instance can be accessed from different parts of the application through the `getInstance()` method.
- **Maintains consistency:** All parts of the application work with the same object and its data, which helps avoid inconsistent states.
- **Useful for shared resources:** It is suitable for resources that should be centrally managed, such as database connections, configuration managers, and logging systems.
- **Simple implementation:** The basic Singleton structure is relatively easy to implement using a private constructor, a static instance, and a public access method.

Cons

- **Global state:** Since the same object can be accessed from many parts of the application, it can behave like a global variable and make the program harder to understand.
- **Difficult to test:** Code that depends directly on a Singleton can be harder to test because it is difficult to replace the single instance with a test or mock object.
- **Can create tight coupling:** Many classes may become directly dependent on the Singleton, making the application less flexible.
- **Multithreading issues:** In a multithreaded application, improper implementation can sometimes create more than one instance if multiple threads access it at the same time.
- **Hidden dependencies:** A class may use the Singleton without clearly showing that it depends on it, making the relationships between different parts of the program less obvious.
- **Not suitable for everything:** Using Singleton unnecessarily can make the design more complicated. It should mainly be used when the application genuinely requires one shared instance.

FACTORY PATTERN

INTRODUCTION:

The Factory Pattern is a creational design pattern that is used to create objects without requiring the user or the main program to know exactly how those objects are created.

Instead of creating objects directly using the new keyword, we use a Factory class or method that decides which type of object should be created based on the given requirement.

For example, in a food-ordering website, a customer may choose different types of food such as Pizza, Burger, or Biryani. Instead of writing object-creation code separately for each food item, a “FoodFactory” can receive the customer's choice and create the appropriate object.

In simple terms:

The Factory Pattern transfers the responsibility of object creation from the main program to a separate Factory.

This makes the code more organized, flexible, and easier to maintain, especially when an application has many different types of related objects.

PROBLEM:

Suppose you are developing a Shape Drawing Application.

You have an interface:

```
interface Shape {  
    void draw();  
}
```

And classes:

```
class Circle implements Shape {}  
class rectangle implements Shape {}  
class square implements Shape {}
```

Client Code:

```
Shape shape = new Circle();  
shape.draw();
```

This works fine initially.

But What happens when the system grows?

Suppose the user choice chooses a shape at runtime.

```
if (type.equals("Circle"))
    shape = new Circle();
else if (type.equals("Rectangle"))
    shape = new Rectangle();
else if (type.equals("Square"))
    shape = new Square();
```

Then the problems are:

(1) Tight Coupling

The client knows all concrete classes.

```
new Circle()
new Rectangle()
new Square()
```

If a class changes, the client code must also change.

2) Code Duplication

The same object creation logic may appear in many places.

```
if (type.equals("Circle"))
```

Repeated again and again.

Design principle:

“A class should be open for extension, but closed for modification.”

A class should allow a design to incorporate new behaviour without modifying existing code.

How to achieve it?

“Identify the aspects of your application that vary and separate them from what stays the same.”

3) Difficult to Extend

Suppose a new shape is added:

```
class Triangle implements Shape
```

Now every client using shape creation logic must be modified.

This violates the Open-Closed Principle.

SOLUTION:

Factory pattern is the solution.

Move the object creation logic into a separate class called factory.

The clients asks the factory for an object.

Structure:

Client



ShapeFactory



Circle — Rectangle — Square

IMPLEMENTATION:

Step 1: Product Interface

```
interface Shape {  
    void draw();  
}
```

Step 2: Concrete Products

```
class Circle implements Shape {  
    public void draw() {  
        System.out.println("Drawing circle!");  
    }  
}
```

Same code for rectangle and square.

Step 3: Factory Class

```
class ShapeFactory {  
    public Shape getShape(String type) {  
        if (type.equalsIgnoreCase("Circle"))  
            return new Circle();  
  
        if (type.equalsIgnoreCase("Rectangle"))  
            return new Rectangle();  
    }  
}
```

```

        if (type.equalsIgnoreCase("Square"))

            return new Square();

        return null;
    }
}

```

The factory decides which object to create.

Step 4: client

```

public class main{

    public static void main(string[] args){

        ShapeFactory factory = new ShapeFactory();

        Shape Shape= factory.getshape("circle");

        Shape.draw();

    }

}

```

OUTPUT:

Drawing circle.

PROS AND CONS OF FACTORY PATTERN:

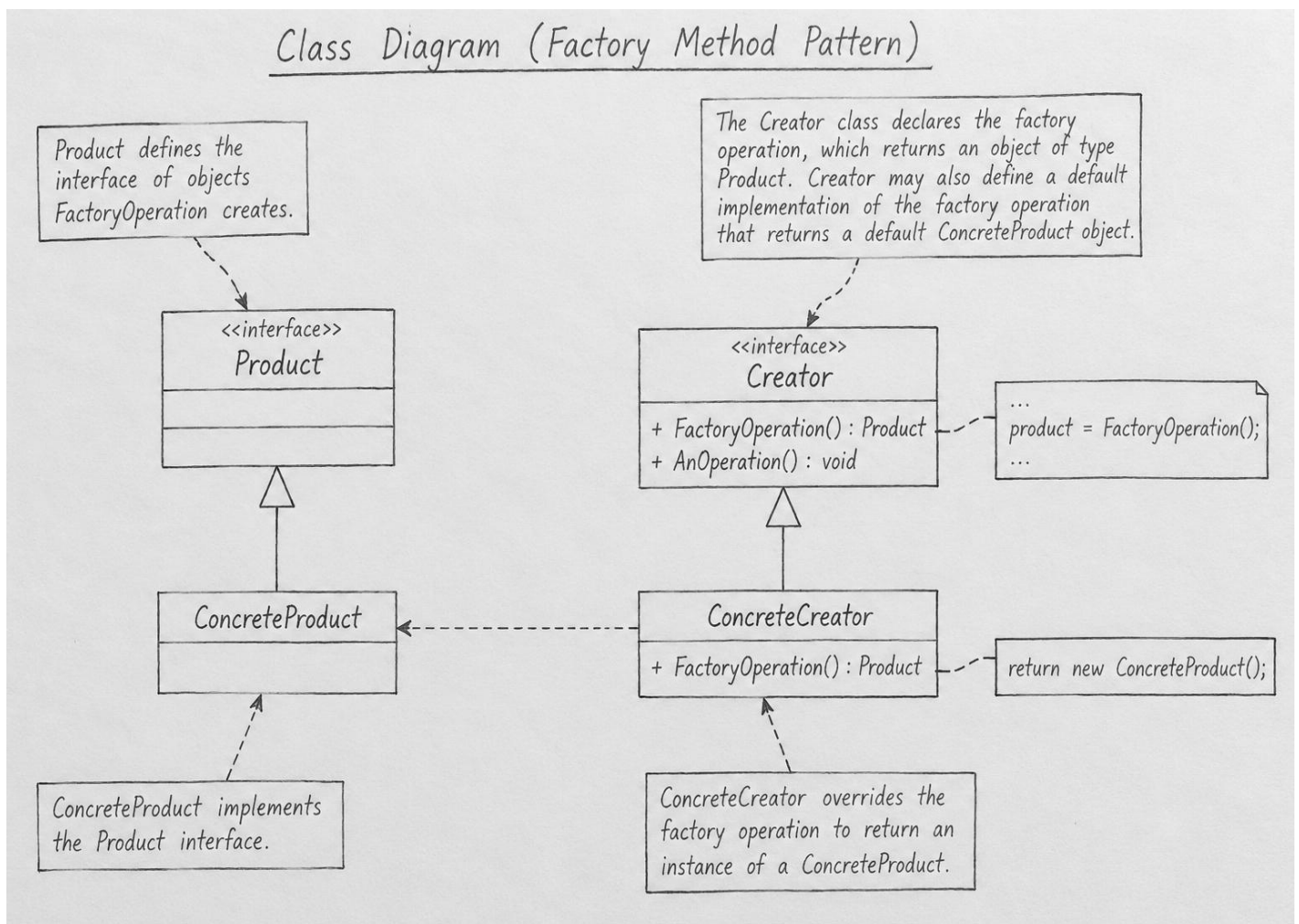
Pros

- **Reduces tight coupling:** The client does not need to directly depend on specific classes like Circle, Rectangle, or Square.
- **Separates object creation:** The responsibility of creating objects is moved to the Factory, keeping the client code clean.
- **Reduces code duplication:** Object-creation logic is written in one central place instead of being repeated throughout the application.
- **Easy to maintain:** If the way an object is created changes, the changes can usually be made inside the Factory without affecting the client code.
- **Easy to extend:** New types of objects can be added without changing the object-creation logic throughout the application.
- **Supports the Open-Closed Principle:** The design can be extended with new types of objects while minimizing modifications to existing client code.

Cons

- **More classes and code:** Using a Factory introduces an additional class, which can make a small application more complex.
- **Factory can become large:** If many types of objects are handled by one Factory, it can contain a large number of if-else or switch statements.
- **Additional abstraction:** For simple applications where only one or two objects are created, using a Factory may be unnecessary.
- **Changes may still be required:** In a basic Factory implementation, adding a completely new type may still require modifying the Factory itself.
- **Slightly harder to understand initially:** Beginners may find it more complicated than directly creating objects with new.

Class diagram:



REAL-LIFE EXAMPLE: Food Ordering System

A good real-life example of the Factory Pattern is a food-ordering application.

Suppose a customer wants to order food. The application can offer different types of food, such as Pizza, Burger, Biryani, or Sandwich. The customer only selects the type of food; they do not need to know how the corresponding food object is created.

A Food Factory handles the creation process. Based on the customer's choice, the factory creates and returns the appropriate food object.

How it works:

Customer selects "Pizza"

↓

FoodFactory

↓

Creates Pizza

↓

Food Object

↓

Order

For example:

- If the customer selects Pizza, the factory creates a Pizza object.
- If the customer selects Burger, the factory creates a Burger object.
- If the customer selects Biryani, the factory creates a Biryani object.

The customer or client code does not need to directly use `new Pizza()`, `new Burger()`, etc. The **Factory takes responsibility for deciding which object to create**.

Why it is useful:

This approach makes the food-ordering system easier to maintain and extend. If a new food item such as Pasta is added, the creation logic can be handled by the factory rather than being repeated throughout the application.

In simple words: The customer chooses *what they want*, while the Factory decides *which object needs to be created*.

OBSERVER PATTERN

INTRODUCTION:

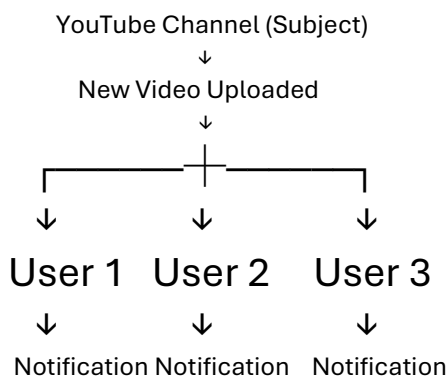
The Observer Pattern is a behavioral design pattern used when one object needs to automatically notify multiple other objects whenever its state or information changes.

In this pattern, there is a Subject that contains some information or state, and one or more Observers that are interested in that information. The observers register themselves with the subject. Whenever something changes in the subject, it automatically notifies all registered observers.

Simple Example

Consider a YouTube channel:

- The YouTube channel is the Subject.
- The Subscribers are the Observers.
- Subscribers subscribe to the channel.
- When the channel uploads a new video, all subscribers are automatically notified.



The main purpose of the Observer Pattern is to create a one-to-many relationship, where a change in one object automatically causes updates or notifications in all objects that depend on it.

PROBLEMS:

Without the Observer Pattern, maintaining communication between one object and multiple dependent objects can become difficult. If the main object's information changes, every dependent object needs to be updated manually.

For example, in a YouTube notification system, if a channel uploads a new video, the channel would have to individually manage and notify every subscriber.

The main problems are:

- **Tight Coupling:** The main object needs to know about all the objects that depend on it.

- **Manual Notifications:** Whenever the subject changes, the programmer may have to manually notify each dependent object.
- **Difficult to Maintain:** As the number of observers increases, the notification code becomes larger and more difficult to manage.
- **Difficult to Extend:** Adding a new observer may require changes to the existing notification logic.
- **Code Duplication:** Similar notification or update code may be repeated in different parts of the application.
- **Poor Scalability:** If thousands of users depend on the same subject, directly managing and notifying each user becomes complicated.

SOLUTION:

The Observer Pattern solves the problem of manually managing and updating multiple objects when the state of one object changes. It allows the main object, called the Subject, to maintain a list of objects called Observers that are interested in its changes.

Whenever the Subject's state changes, it automatically sends an update to all registered Observers. The Subject does not need to know the internal details of each Observer; it only needs to notify them through a common interface.

For example, in a YouTube notification system, users can subscribe to a channel. When the channel uploads a new video, the channel automatically notifies all its subscribers. If a user unsubscribes, they can simply be removed from the list of Observers.

This approach provides several advantages:

- **Loose coupling:** The Subject and Observers are not strongly dependent on each other's implementation.
- **Automatic notification:** Observers are automatically informed whenever the Subject changes.
- **Easy to extend:** New observers can be added without changing the existing notification mechanism.
- **Better maintainability:** Notification logic is organized in one place.
- **Dynamic subscription:** Observers can subscribe or unsubscribe whenever required.

REAL-LIFE EXAMPLE: YouTube Notification System

A common real-life example of the Observer Pattern is the notification system of a YouTube channel.

When a user subscribes to a YouTube channel, they become an observer of that channel. The channel acts as the subject. Whenever the channel uploads a new video, all subscribed users are automatically notified.

For example, if a channel has 1,000 subscribers, the channel does not need to individually manage what each subscriber should do. Instead, it sends an update, and the notification system informs all registered subscribers.

Users can also unsubscribe at any time. Once they unsubscribe, they are removed from the list and will no longer receive notifications.

This is useful because the channel and its subscribers remain loosely connected, while subscribers can be added or removed dynamically.

IMPLEMENTATION:

The Observer Pattern can be implemented by creating a Subject and an Observer interface. The Subject maintains a list of registered observers and notifies them whenever its state changes.

For the YouTube example, the YouTube Channel acts as the Subject, while the Subscribers act as Observers.

Java Implementation:

```
import java.util.ArrayList;

import java.util.List;

// Observer Interface

interface Observer {

    void update(String videoTitle);

}

// Concrete Observer

class Subscriber implements Observer {

    private String name;

    public Subscriber(String name) {

        this.name = name;

    }

}
```



```

@Override

public void update(String videoTitle) {

    System.out.println(

        name + " received notification: New video uploaded - "

        + videoTitle

    );

}

}

```

// Subject

```

class YouTubeChannel {

    private List<Observer> subscribers = new ArrayList<>();

    private String videoTitle;

    // Register an observer

    public void subscribe(Observer observer) {

        subscribers.add(observer);

    }

    // Remove an observer

    public void unsubscribe(Observer observer) {

        subscribers.remove(observer);

    }

    // Notify all observers

    private void notifySubscribers() {

        for (Observer subscriber : subscribers) {

            subscriber.update(videoTitle);

        }

    }

}

// Change the state of the Subject

```

```

public void uploadVideo(String videoTitle) {
    this.videoTitle = videoTitle;
    System.out.println(
        "\nNew video uploaded: " + videoTitle
    );
    notifySubscribers();
}
}

// Main class

public class Main {
    public static void main(String[] args) {
        YouTubeChannel channel = new YouTubeChannel();
        Subscriber user1 = new Subscriber("Praveen");
        Subscriber user2 = new Subscriber("Rahul");
        Subscriber user3 = new Subscriber("Aman");
        // Users subscribe to the channel
        channel.subscribe(user1);
        channel.subscribe(user2);
        channel.subscribe(user3);
        // Upload a new video
        channel.uploadVideo("Java Design Patterns");
        // User2 unsubscribes
        channel.unsubscribe(user2);
        // Upload another video
        channel.uploadVideo("Observer Pattern Explained");
    }
}

```

Output:

New video uploaded: Java Design Patterns

Praveen received notification: New video uploaded - Java Design Patterns

Rahul received notification: New video uploaded - Java Design Patterns

Aman received notification: New video uploaded - Java Design Patterns

New video uploaded: Observer Pattern Explained

Praveen received notification: New video uploaded - Observer Pattern Explained

Aman received notification: New video uploaded - Observer Pattern Explained

How the Implementation Works:

The Observer interface defines the `update()` method that every subscriber must implement. The Subscriber class implements this interface and defines what should happen when it receives a notification.

The YouTubeChannel class acts as the Subject. It maintains a list of subscribers and provides methods to subscribe and unsubscribe users.

When a new video is uploaded using `uploadVideo()`, the channel changes its state and calls `notifySubscribers()`. This method goes through all registered subscribers and calls their `update()` method.

When user2 unsubscribes, they are removed from the subscriber list. Therefore, they do not receive notifications for future videos.

This implementation demonstrates how the Observer Pattern allows multiple objects to automatically receive updates when the state of another object changes.

PROS AND CONS OF OBSERVER PATTERN:

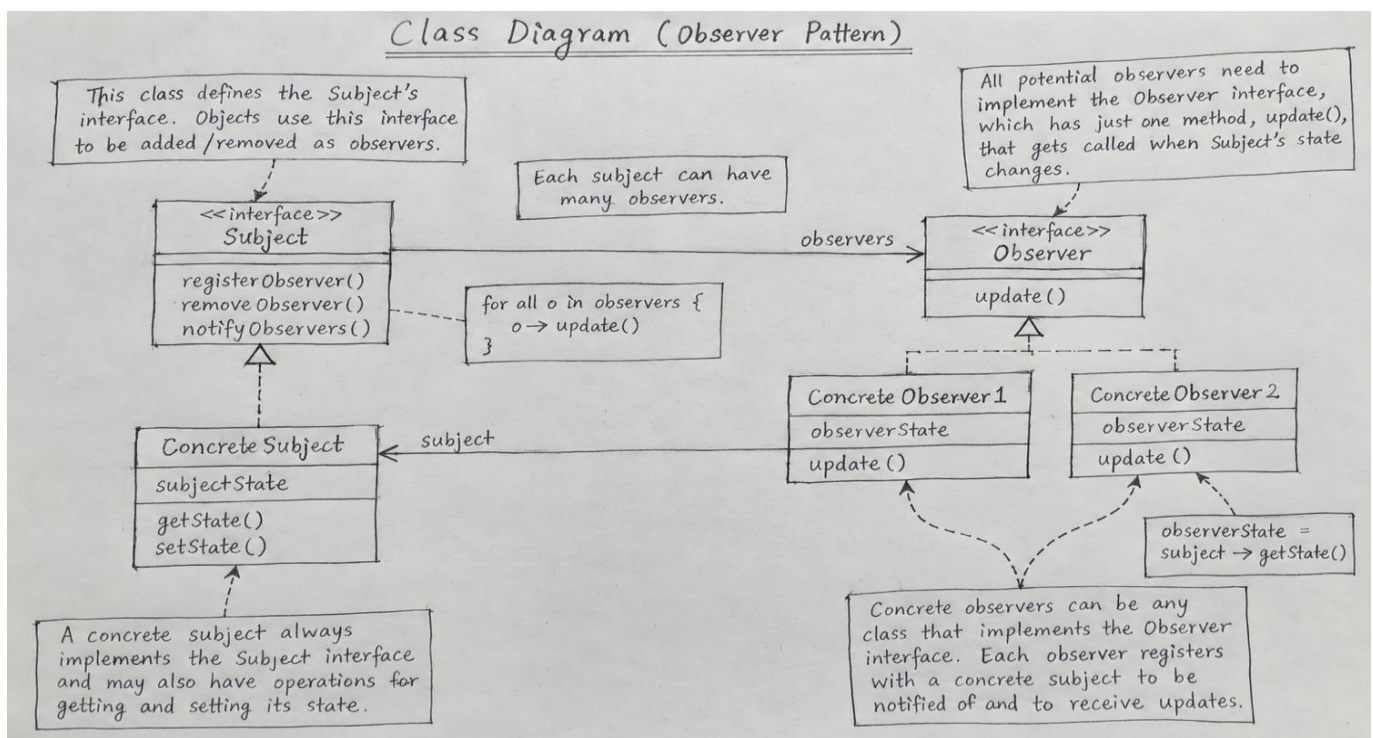
Pros

- **Loose coupling:** The Subject does not need to know the detailed implementation of its Observers.
- **Automatic updates:** Observers are automatically notified whenever the Subject's state changes.
- **Easy to extend:** New Observers can be added without making major changes to the Subject.
- **Dynamic subscription:** Observers can subscribe or unsubscribe at any time according to their requirements.
- **Better maintainability:** The notification and update mechanism is organized and separated from the main business logic.
- **Supports one-to-many communication:** A single Subject can notify multiple Observers at the same time.

Cons

- **Too many notifications:** If there are many Observers, a single change in the Subject can trigger a large number of updates.
- **Performance issues:** Notifying a large number of Observers can increase processing time and affect application performance.
- **Difficult to track updates:** In a complex system, it can be difficult to determine which Observer received an update and what action it performed.
- **Unexpected updates:** An Observer may receive notifications even when the change is not important to it.
- **Memory management:** If Observers are not properly removed or unsubscribed, they may remain connected to the Subject and unnecessarily consume memory.
- **Debugging can be difficult:** Since updates happen automatically, it may be harder to trace the exact sequence of events when something goes wrong.

CLASS DIAGRAM:



STRATEGY PATTERN

INTRODUCTION:

The Strategy Pattern is a behavioural design pattern that allows us to define multiple ways of performing a particular task and choose one of them when needed.

Instead of putting all possible algorithms or behaviours inside a single class using many if-else or switch statements, each behaviour is placed in a separate strategy class. The required strategy can then be selected at runtime.

For example, in an online payment system, a customer may have different payment options such as UPI, Credit Card, or PayPal. Each payment method works differently, so we can create a separate strategy for each method.

The application can then select the required payment strategy based on the customer's choice.

In simple words: The Strategy Pattern allows us to change the way a task is performed without changing the main class that uses it.

PROBLEMS:

Without the Strategy Pattern, an application may place different algorithms or ways of performing a task inside one class. As the number of options increases, the code can become large, complicated, and difficult to maintain.

For example, in an online payment system, the application may support UPI, Credit Card, Debit Card, and PayPal. Without the Strategy Pattern, the code might contain many if-else or switch statements to decide which payment method to use.

The main problems are:

- **Large if-else or switch statements:** The class may contain separate conditions for every available strategy.
- **Tight coupling:** The main class becomes directly dependent on all the different algorithms or payment methods.
- **Difficult to maintain:** Changes to one payment method may require modifications to the main class.
- **Difficult to extend:** Adding a new strategy, such as Net Banking, may require adding more conditions to existing code.
- **Code duplication:** Similar logic may be repeated for different strategies.

- **Violation of the Open-Closed Principle:** The main class needs to be modified whenever a new behaviour or algorithm is added.

The problem is that putting multiple ways of performing the same task into one class makes the code complex, tightly coupled, and difficult to change or extend.

SOLUTION:

The Strategy Pattern solves the problem by separating different algorithms or behaviors into individual classes instead of putting everything inside one class.

A common Strategy interface is created that defines the operation to be performed. Each different behavior then implements this interface as a separate strategy. The main class, called the Context, uses the selected strategy without needing to know how that strategy works internally.

For example, in an online payment system, instead of putting UPI, Credit Card, Debit Card, and PayPal logic inside one class, we can create separate payment strategies for each method.

How it solves the problems:

- **Removes complex conditions:** Different strategies are placed in separate classes, reducing large if-else or switch statements.
- **Reduces coupling:** The Context depends on the common Strategy interface rather than directly depending on every payment method.
- **Easy to maintain:** Changes to one strategy can be made in its own class without affecting the other strategies.
- **Easy to extend:** A new payment method can be added by creating a new strategy class without changing the existing strategies.
- **Avoids code duplication:** Each algorithm or behavior has its own dedicated implementation.
- **Supports runtime selection:** The required strategy can be selected or changed while the program is running.

Instead of putting all possible ways of doing a task into one class, we separate each way into its own class and allow the application to choose the required strategy.

REAL-LIFE EXAMPLE: Online Payment System

A good real-life example of the Strategy Pattern is an online payment system.

Suppose an e-commerce website allows customers to pay using different methods such as UPI, Credit Card, Debit Card, or Net Banking. Each payment method follows a different process, but the overall goal is the same: make the payment.

Instead of writing all payment methods inside one class using multiple if-else conditions, each payment method can be treated as a separate strategy.

For example:

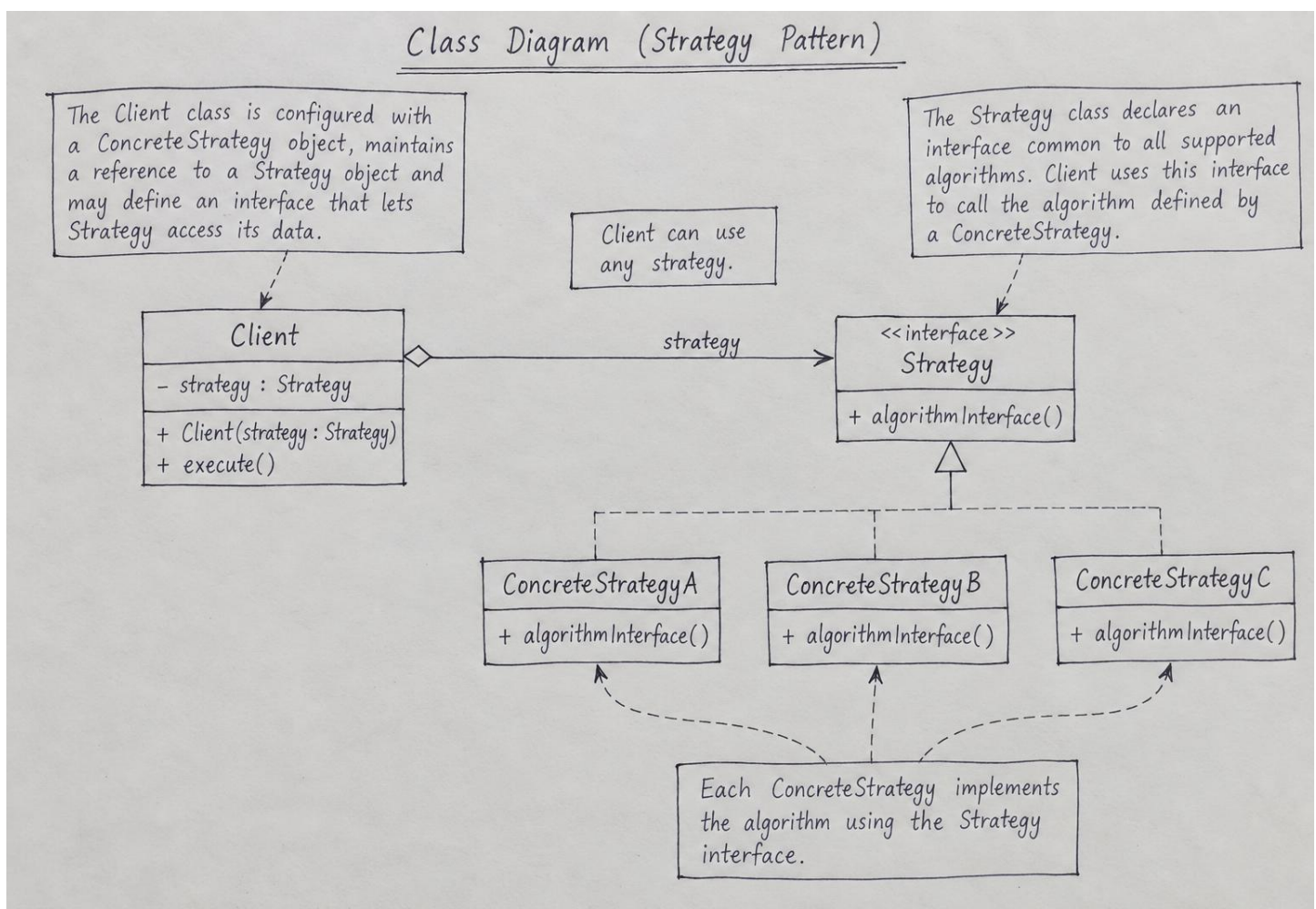
- **UPI Strategy:** handles UPI payments.
- **Credit Card Strategy:** handles credit card payments.
- **Debit Card Strategy:** handles debit card payments.
- **Net Banking Strategy:** handles net banking payments.

The customer can select the required payment method, and the application uses that strategy to process the payment.

Why Strategy Pattern is useful here:

If the website later adds another payment method, such as a Digital Wallet, a new strategy can be created without changing the existing payment strategies.

CLASS DIAGRAM:



IMPLEMENTATION:

The Strategy Pattern can be implemented by creating a common Strategy interface and then creating separate classes for each strategy. The main Context class keeps a reference to the Strategy and uses it to perform the required operation.

For our online payment example, we can create different strategies such as UPI, Credit Card, and Net Banking. Each strategy implements the same PaymentStrategy interface but provides its own way of processing the payment.

Java Implementation:

// Strategy Interface

```
interface PaymentStrategy {  
    void pay(double amount);  
}
```

// Concrete Strategy 1

```
class UPIPayment implements PaymentStrategy {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Payment of ₹" + amount  
            + " made using UPI.");  
    }  
}
```

// Concrete Strategy 2

```
class CreditCardPayment implements PaymentStrategy {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Payment of ₹" + amount  
            + " made using Credit Card.");  
    }  
}
```

// Concrete Strategy 3

```
class NetBankingPayment implements PaymentStrategy {
```



```

@Override

public void pay(double amount) {

    System.out.println("Payment of ₹" + amount
        + " made using Net Banking.");
}

}

// Context

class PaymentContext {

    private PaymentStrategy strategy;

    // Set the required strategy

    public void setStrategy(PaymentStrategy strategy) {

        this.strategy = strategy;
    }

    // Execute the selected strategy

    public void makePayment(double amount) {

        if (strategy != null) {

            strategy.pay(amount);

        } else {

            System.out.println("Please select a payment method.");

        }

    }

}

// Main Class

public class Main {

    public static void main(String[] args) {

        PaymentContext payment = new PaymentContext();

        // Select UPI strategy

        payment.setStrategy(new UPIPayment());

        payment.makePayment(1500);
    }
}

```

```

// Change to Credit Card strategy
payment.setStrategy(new CreditCardPayment());
payment.makePayment(2500);
// Change to Net Banking strategy
payment.setStrategy(new NetBankingPayment());
payment.makePayment(3000);
}
}

```

Output:

Payment of ₹1500.0 made using UPI.

Payment of ₹2500.0 made using Credit Card.

Payment of ₹3000.0 made using Net Banking.

How the Implementation Works:

The PaymentStrategy interface defines the common pay() method that all payment strategies must implement.

The UPIPayment, CreditCardPayment, and NetBankingPayment classes are **Concrete Strategies**. **Each class provides its own implementation of the pay() method.**

The PaymentContext class maintains a reference to the PaymentStrategy. Using setStrategy(), the required payment method can be selected or changed at runtime.

The important point is that the PaymentContext does not need to know how UPI, Credit Card, or Net Banking payments are processed. It only calls the common pay() method.

Pros and Cons of Strategy Pattern:

Pros

- **Reduces complex conditions:** It avoids large if-else or switch statements by placing different algorithms in separate classes.
- **Loose coupling:** The Context depends on the common Strategy interface instead of being directly connected to every specific strategy.
- **Easy to extend:** New strategies can be added without changing the existing strategies or the main Context class.
- **Easy to maintain:** Each strategy has its own class, so changes can be made independently.

- **Supports runtime selection:** The required strategy can be selected or changed while the application is running.
- **Promotes code reusability:** The same strategy can be used by different Contexts or parts of an application.
- **Follows the Open-Closed Principle:** New behaviors can be added by creating new strategy classes rather than modifying existing code.

Cons

- **More classes:** Each strategy usually requires a separate class, which increases the number of classes in the application.
- **Increased complexity:** For a simple task with only one or two possible behaviors, using the Strategy Pattern may be unnecessary.
- **Client needs to choose the strategy:** The application or client must know which strategy is appropriate for the given situation.
- **Communication overhead:** The Context and Strategy may need to exchange additional information, which can make the design slightly more complicated.
- **Difficult to understand initially:** Beginners may find it harder to understand because the actual behavior is separated across multiple classes.
- **Too many strategies can become difficult to manage:** If an application has a very large number of strategies, selecting and maintaining them can become complicated.

DECORATOR PATTERN

INTRODUCTION:

The Decorator Pattern is a structural design pattern used to add new features or responsibilities to an existing object without changing its original class.

Instead of modifying the original class or creating many subclasses for every possible combination of features, we can wrap the original object with one or more decorators. Each decorator adds some extra functionality while keeping the original object unchanged.

For example, consider a coffee shop application. A basic coffee can be ordered first, and customers can add extra features such as milk, sugar, whipped cream, or chocolate. Instead of creating separate classes like `CoffeeWithMilk`, `CoffeeWithMilkAndSugar`, and `CoffeeWithMilkSugarAndChocolate`, decorators can be used to add these features dynamically.

The Decorator Pattern allows us to add extra features to an existing object dynamically without modifying its original class.

PROBLEMS:

Without the Decorator Pattern, adding new features to an existing object can make the code complex, difficult to maintain, and less flexible.

For example, in a coffee shop application, suppose we have a basic `Coffee` class. Customers can add milk, sugar, chocolate, whipped cream, or other toppings. If we try to handle every possible combination using separate classes, the number of classes can increase quickly.

The main problems are:

- **Too many subclasses:** A separate class may be required for every combination of additional features.
- **Class explosion:** As more features are added, the number of possible combinations grows, resulting in a large number of classes.
- **Difficult to maintain:** Changes or additions to features may require modifications to many existing classes.
- **Inflexible design:** Features are often fixed when the object is created, making it difficult to add or remove them dynamically.
- **Code duplication:** Similar functionality may be repeated across different subclasses.
- **Violates the Open-Closed Principle:** Adding new features may require modifying existing classes instead of simply extending the system.

SOLUTION:

The Decorator Pattern solves these problems by allowing us to add new features or responsibilities to an object without modifying its original class.

Instead of creating a separate subclass for every possible combination of features, we create decorator classes that wrap the original object. Each decorator adds one specific feature and can be combined with other decorators as needed.

For example, in a coffee shop application, we can start with a basic Coffee object. If the customer wants milk, we wrap the coffee with a MilkDecorator. If they also want sugar, we can wrap it again with a SugarDecorator. More features can be added in the same way.

How it solves the problems:

- **Avoids too many subclasses:** We don't need a separate class for every combination of features.
- **Adds features dynamically:** Features can be added to an object at runtime according to the user's requirements.
- **Keeps the original class unchanged:** The original Coffee class does not need to be modified when a new topping is introduced.
- **Reduces code duplication:** Each decorator contains the logic for its own feature and can be reused.
- **Supports flexible combinations:** Multiple decorators can be combined in different ways to create different combinations of features.
- **Easy to extend:** New features can be added by creating new decorator classes without changing existing classes.

Instead of creating a new class for every combination of features, we wrap the existing object with decorators, and each decorator adds one extra feature.

Real-Life Example: Coffee Ordering System

Suppose a customer first orders a basic coffee. They can then add extra features such as milk, sugar, chocolate, whipped cream, or caramel according to their preference.

Instead of creating a separate class for every possible combination, such as CoffeeWithMilk, CoffeeWithMilkAndSugar, or CoffeeWithMilkSugarAndChocolate, we can use decorators to add these features one by one.

For example:

- Coffee → Basic coffee
- MilkDecorator → Adds milk

- SugarDecorator → Adds sugar
- ChocolateDecorator → Adds chocolate
- WhippedCreamDecorator → Adds whipped cream

A customer can combine these decorators in any order depending on what they want.

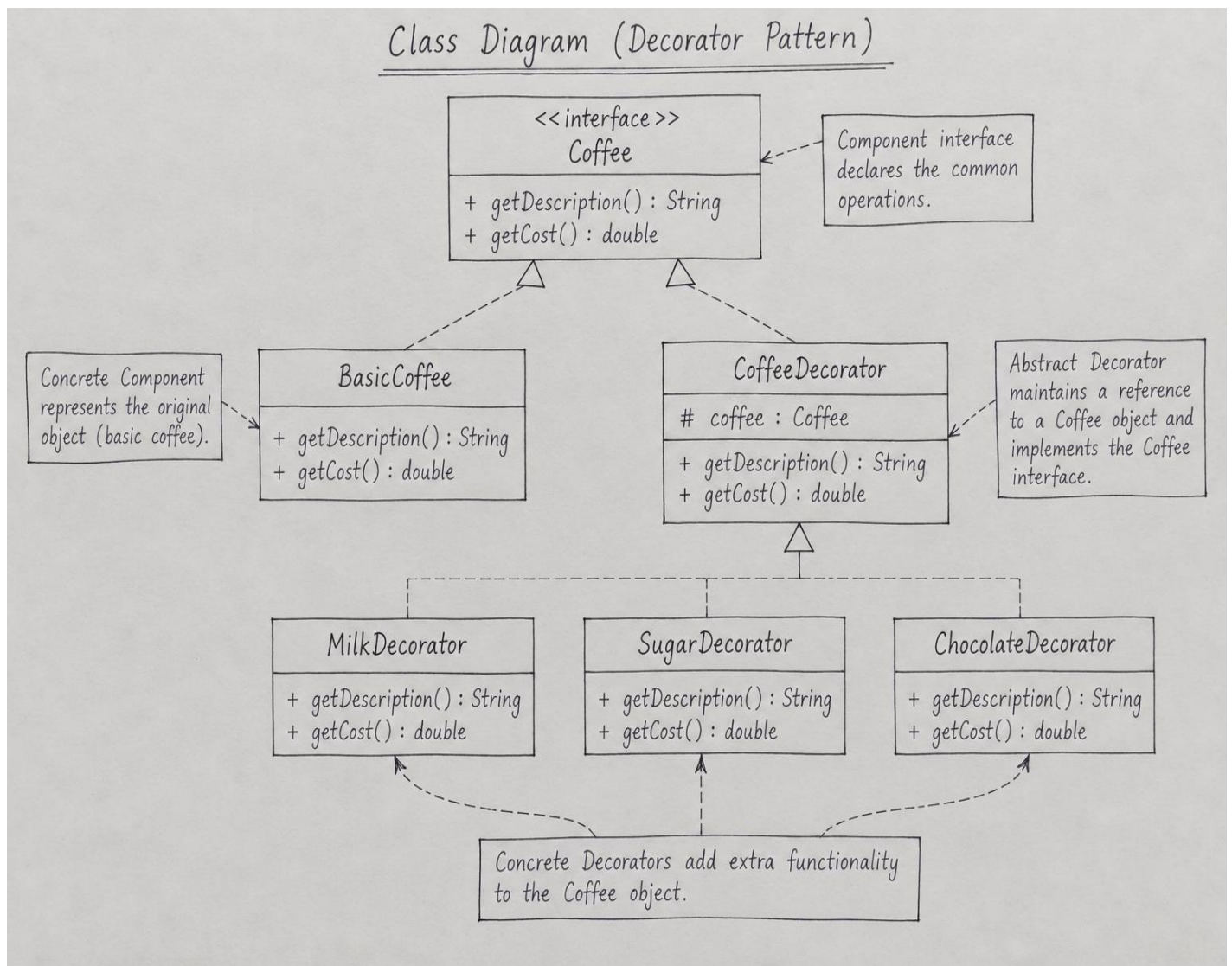
Example:

A customer orders:

Basic Coffee + Milk + Sugar + Chocolate

The original coffee object remains unchanged, while each decorator adds its own feature to it.

CLASS DIAGRAM:



IMPLEMENTATION:

The Decorator Pattern can be implemented by creating a common interface, a basic concrete component, and one or more decorator classes. The decorators wrap the original object and add extra functionality to it without changing the original class.

For our coffee ordering system, we will use Coffee as the basic component. Customers can then add milk, sugar, and chocolate using different decorators.

Java Implementation:

// Component Interface

```
interface Coffee {  
    String getDescription();  
    double getCost();  
}
```

// Concrete Component

```
class BasicCoffee implements Coffee {  
    @Override  
    public String getDescription() {  
        return "Basic Coffee";  
    }  
    @Override  
    public double getCost() {  
        return 50.0;  
    }  
}
```

// Abstract Decorator

```
abstract class CoffeeDecorator implements Coffee {  
    protected Coffee coffee;  
    public CoffeeDecorator(Coffee coffee) {  
        this.coffee = coffee;  
    }  
    @Override
```

```

public String getDescription() {
    return coffee.getDescription();
}

@Override
public double getCost() {
    return coffee.getCost();
}

// Concrete Decorator - Milk
class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + " + Milk";
    }

    @Override
    public double getCost() {
        return coffee.getCost() + 20.0;
    }
}

// Concrete Decorator - Sugar
class SugarDecorator extends CoffeeDecorator {
    public SugarDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {

```



```

        return coffee.getDescription() + " + Sugar";
    }

    @Override
    public double getCost() {
        return coffee.getCost() + 10.0;
    }
}

// Concrete Decorator - Chocolate
class ChocolateDecorator extends CoffeeDecorator {
    public ChocolateDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + " + Chocolate";
    }

    @Override
    public double getCost() {
        return coffee.getCost() + 30.0;
    }
}

// Main Class
public class Main {
    public static void main(String[] args) {
        // Create basic coffee
        Coffee coffee = new BasicCoffee();

        System.out.println("Order: " + coffee.getDescription());

        System.out.println("Price: ₹" + coffee.getCost());

        // Add Milk

```

```

    coffee = new MilkDecorator(coffee);
    // Add Sugar
    coffee = new SugarDecorator(coffee);
    // Add Chocolate
    coffee = new ChocolateDecorator(coffee);
    System.out.println("\nFinal Order: "
        + coffee.getDescription());
    System.out.println("Final Price: ₹"
        + coffee.getCost());
}
}

```

Output:

Order: Basic Coffee

Price: ₹50.0

Final Order: Basic Coffee + Milk + Sugar + Chocolate

Final Price: ₹110.0

How the Implementation Works:

The Coffee interface defines the common operations `getDescription()` and `getCost()`.

The BasicCoffee class provides the basic coffee and its original price.

The CoffeeDecorator class acts as the base decorator. It contains a reference to a Coffee object, allowing it to wrap another coffee object.

The MilkDecorator, SugarDecorator, and ChocolateDecorator are concrete decorators. Each one adds its own feature and cost while keeping the original BasicCoffee class unchanged.

The decorators can also be combined together. In the example, milk is added first, followed by sugar and chocolate. Each decorator receives the previously decorated object and adds its own functionality.

We start with a basic coffee and wrap it with different decorators to add extra features. This allows us to add or combine features dynamically without modifying the original coffee class.

PROS AND CONS OF DECORATOR PATTERN:

Pros

- **Adds features dynamically:** New functionality can be added to an object at runtime without changing its original class.
- **Avoids class explosion:** It reduces the need to create many subclasses for different combinations of features.
- **Flexible design:** Multiple decorators can be combined in different ways to create different combinations of features.
- **Follows the Open-Closed Principle:** Existing classes remain unchanged while new functionality can be added through new decorators.
- **Promotes code reusability:** A decorator can be reused with different objects that follow the same interface.
- **Keeps responsibilities separate:** Each decorator is responsible for adding a specific feature, making the code easier to organize.

Cons

- **Many small classes:** Each additional feature may require a separate decorator class, increasing the number of classes.
- **Can become complex:** Using many decorators together can make the structure of the program difficult to understand.
- **Difficult debugging:** When several decorators are wrapped around an object, it can be harder to identify where a particular behavior comes from.
- **Order can matter:** Applying decorators in different orders may produce different results, especially when decorators modify the same behavior.
- **Object creation can become complicated:** Creating and combining several layers of decorators may make the code less readable.
- **Extra processing:** Each decorator adds another layer through which method calls must pass, which can introduce a small amount of overhead.

The Decorator Pattern provides great flexibility for adding features, but using too many decorators can make the code complex and difficult to understand or debug.

COMBINE REAL LIFE EXAMPLE OF ALL 5 DESIGN PATTERNS:

Food Ordering Application:

A food ordering application is a good real-life example where all five design patterns can work together. Each pattern handles a different part of the application and solves a specific design problem.

1. Singleton Pattern – Database Connection

The application can use the Singleton Pattern to manage the database connection.

There should be only one shared database connection manager used by different parts of the application, such as user management, restaurants, orders, and payments.

Example: `DatabaseConnection.getInstance()`

This prevents unnecessary creation of multiple connection objects.

2. Factory Pattern – Creating Food Items

The Factory Pattern can be used when the customer selects a type of food.

For example, the customer can choose:

- Pizza
- Burger
- Biryani
- Sandwich

Instead of the application directly creating every object, a `FoodFactory` creates the appropriate food object based on the customer's choice.

Example: `FoodFactory.createFood("Pizza")`

This keeps the object-creation logic separate from the main application.

3. Observer Pattern – Order Notifications

The Observer Pattern can be used for order status notifications.

When an order is placed, the customer can be registered as an observer of that order. Whenever the order status changes, the customer is automatically notified.

For example:

Order Placed → Food Preparing → Out for Delivery → Delivered

The customer does not need to repeatedly check the application because updates are automatically sent whenever the order status changes.

4. Strategy Pattern – Payment Methods

The Strategy Pattern can be used to handle different payment methods.

The application can support:

- UPI
- Credit Card
- Debit Card
- Net Banking

Each payment method is implemented as a separate strategy. The customer can select the required payment method at runtime.

Example: UPIPayment, CreditCardPayment, NetBankingPayment

The main payment system does not need to contain complicated if-else conditions for every payment method.

5. Decorator Pattern – Customizing Food

The Decorator Pattern can be used when customers want to add extra features to their food.

For example, a customer orders a basic pizza and then adds:

- Extra Cheese
- Olives
- Mushrooms
- Extra Sauce

Instead of creating a separate class for every possible combination, decorators can dynamically add these features to the food object.

Example: Pizza + ExtraCheese + Olives + Mushrooms

How All Five Work Together:

Consider a customer ordering a customized pizza through the application:

1. **Factory Pattern** creates the Pizza object based on the customer's choice.

2. **Decorator Pattern** adds extra cheese and other toppings to the pizza.
3. **Strategy Pattern** processes the payment using the customer's selected payment method, such as UPI.
4. **Observer Pattern** automatically sends order-status updates to the customer.
5. **Singleton Pattern** provides a single shared database connection for storing and retrieving the order information.

Overall Example:

Food Ordering Application

- Factory creates the food
- Decorator adds extra features
- Strategy processes payment
- Observer sends order updates
- Singleton manages the shared database connection

This demonstrates how different design patterns can be combined in one real-world application, with each pattern having a specific responsibility.

IMPLEMENTAION OF COMBINED DESIGN PATTERN:

A Food Ordering Application can demonstrate all five design patterns together. Each pattern is responsible for a different part of the system.

1. Singleton Pattern – Database Connection

The Singleton Pattern ensures that only one database connection object is created and shared throughout the application.

```
class DatabaseConnection
```

```
    private static DatabaseConnection instance;

    private DatabaseConnection() {
        System.out.println("Database Connection Created");
    }

    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    public void saveOrder(String order) {
        System.out.println("Order saved: " + order);
    }
}
```

```
}  
  
}
```

Here, the constructor is private, so other classes cannot create a new database connection directly. The getInstance() method always returns the same object.

2. Factory Pattern – Creating Food

The Factory Pattern is used to create different types of food without making the main application responsible for object creation.

```
interface Food {  
    void prepare();  
}  
  
class Pizza implements Food {  
    public void prepare() {  
        System.out.println("Pizza is being prepared.");  
    }  
}  
  
class Burger implements Food {  
    public void prepare() {  
        System.out.println("Burger is being prepared.");  
    }  
}  
  
class Biryani implements Food {  
    public void prepare() {  
        System.out.println("Biryani is being prepared.");  
    }  
}  
  
class FoodFactory {  
    public static Food createFood(String type) {  
        if (type.equalsIgnoreCase("Pizza")) {  
            return new Pizza();  
        } else if (type.equalsIgnoreCase("Burger")) {  
            return new Burger();  
        } else if (type.equalsIgnoreCase("Biryani")) {
```

```

        return new Biryani();
    }

    return null;
}
}

```

Now the client can simply request a food item:

```

Food food = FoodFactory.createFood("Pizza");

food.prepare();

```

3. Decorator Pattern – Adding Extra Toppings

The Decorator Pattern allows customers to add extra features to their food without modifying the original food class.

```

interface FoodItem {

    String getDescription();

    double getPrice();

}

class BasicPizza implements FoodItem {

    public String getDescription() {

        return "Basic Pizza";

    }

    public double getPrice() {

        return 200;

    }

}

abstract class PizzaDecorator implements FoodItem {

    protected FoodItem pizza;

    public PizzaDecorator(FoodItem pizza) {

        this.pizza = pizza;

    }

    public String getDescription() {

        return pizza.getDescription();

    }

    public double getPrice() {

        return pizza.getPrice();

    }

}

```



```

    }
}

class ExtraCheese extends PizzaDecorator {

    public ExtraCheese(FoodItem pizza) {

        super(pizza);

    }

    public String getDescription() {

        return pizza.getDescription() + " + Extra Cheese";

    }

    public double getPrice() {

        return pizza.getPrice() + 50;

    }

}

class Mushrooms extends PizzaDecorator {

    public Mushrooms(FoodItem pizza) {

        super(pizza);

    }

    public String getDescription() {

        return pizza.getDescription() + " + Mushrooms";

    }

    public double getPrice() {

        return pizza.getPrice() + 40;

    }

}

```

The customer can customize the pizza:

```

FoodItem pizza = new BasicPizza();

pizza = new ExtraCheese(pizza);

pizza = new Mushrooms(pizza);

System.out.println(pizza.getDescription());

System.out.println("Price: ₹" + pizza.getPrice());

```

The same pizza object can be dynamically decorated with different toppings.

4. Strategy Pattern – Payment Method

The Strategy Pattern is used to provide different payment methods.

```
interface PaymentStrategy {  
    void pay(double amount);  
}  
  
class UPIPayment implements PaymentStrategy {  
    public void pay(double amount) {  
        System.out.println("Payment of ₹" + amount  
            + " made using UPI.");  
    }  
}  
  
class CardPayment implements PaymentStrategy {  
    public void pay(double amount) {  
        System.out.println("Payment of ₹" + amount  
            + " made using Credit Card.");  
    }  
}  
  
class PaymentContext {  
    private PaymentStrategy strategy;  
    public void setStrategy(PaymentStrategy strategy) {  
        this.strategy = strategy;  
    }  
    public void makePayment(double amount) {  
        strategy.pay(amount);  
    }  
}
```

The customer can select the payment method at runtime:

```
PaymentContext payment = new PaymentContext();
```

```
payment.setStrategy(new UPIPayment());
```

```
payment.makePayment(290);
```

The payment method can also be changed without modifying the PaymentContext:

```
payment.setStrategy(new CardPayment());
```

```
payment.makePayment(290);
```

5. Observer Pattern – Order Notifications

The Observer Pattern is used to automatically notify the customer whenever the order status changes.

```
import java.util.ArrayList;

import java.util.List;

interface Observer {

    void update(String status);

}

class Customer implements Observer {

    private String name;

    public Customer(String name) {

        this.name = name;

    }

    public void update(String status) {

        System.out.println(

            name + " received notification: " + status

        );

    }

}

class Order {

    private List<Observer> customers = new ArrayList<>();

    public void addObserver(Observer customer) {

        customers.add(customer);

    }

    public void removeObserver(Observer customer) {

        customers.remove(customer);

    }

    public void updateStatus(String status) {

        System.out.println("Order Status: " + status);

        for (Observer customer : customers) {

            customer.update(status);

        }

    }

}
```

```
}  
}
```

The customer can register for order updates:

```
Order order = new Order();  
  
Customer customer = new Customer("Praveen");  
  
order.addObserver(customer);  
  
order.updateStatus("Order Placed");  
  
order.updateStatus("Food is Being Prepared");  
  
order.updateStatus("Out for Delivery");
```

The customer automatically receives every update.

Main Program:

All five patterns can then be used together in the main application:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        // Singleton  
  
        DatabaseConnection db =  
  
            DatabaseConnection.getInstance();  
  
        // Factory  
  
        Food food =  
  
            FoodFactory.createFood("Pizza");  
  
        food.prepare();  
  
        // Decorator  
  
        FoodItem pizza =  
  
            new BasicPizza();  
  
        pizza = new ExtraCheese(pizza);  
  
        pizza = new Mushrooms(pizza);  
  
        System.out.println(  
  
            "Order: " + pizza.getDescription()  
  
        );  
  
        System.out.println(  
  
            "Total Price: ₹" + pizza.getPrice()  
  
        );  
    }  
}
```

```

// Strategy

PaymentContext payment =
    new PaymentContext();

payment.setStrategy(new UPIPayment());

payment.makePayment(pizza.getPrice());

// Observer

Order order = new Order();

Customer customer =
    new Customer("Praveen");

order.addObserver(customer);

order.updateStatus("Order Placed");

order.updateStatus("Food is Being Prepared");

order.updateStatus("Out for Delivery");

// Singleton database

db.saveOrder(pizza.getDescription());

}

}

```

Sample Output:

Database Connection Created

Pizza is being prepared.

Order: Basic Pizza + Extra Cheese + Mushrooms

Total Price: ₹290.0

Payment of ₹290.0 made using UPI.

Order Status: Order Placed

Praveen received notification: Order Placed

Order Status: Food is Being Prepared

Praveen received notification: Food is Being Prepared

Order Status: Out for Delivery

Praveen received notification: Out for Delivery

Order saved: Basic Pizza + Extra Cheese + Mushrooms

How the Five Patterns Work Together:

In this implementation:

- **Singleton** → manages the single database connection.

- **Factory** → creates the required food object.
- **Decorator** → adds extra toppings to the food dynamically.
- **Strategy** → allows the customer to select a payment method.
- **Observer** → automatically sends order-status notifications.

This shows that the five patterns are not just theoretical concepts; they can work together to build a flexible and maintainable real-world food-ordering application.